UNIVERSIDAD DE LOS LLANOS
Facultad de Ciencias Básicas e Ingeniería
Programa de Ingeniería de Sistemas

MANUAL TÉCNICO

Sistema de optimización de la red logística de recepción y procesamiento de residuos agropecuarios

EcoResiduos del Llano S.A.S.

Autores
Lucia A. Vega — Líder de proyecto / Analista de negocio
Emmanuel F. Camacho — Ingeniero de modelado y optimización
Astrid D. Martinez — Desarrolladora de metaheurísticas
Samuel D. Martinez — Desarrollador de software e interfaz

Contenido

1. Introducción 3
2. Descripción general 3
3. Arquitectura 3
4. Requisitos 4
5. Instalación y ejecución 5
5.1 Obtener el código 5
5.2 Backend (API) 5
5.3 Frontend (interfaz) 5
5.4 Variables de entorno 5
5.5 Base de datos (opcional) 5
6. Estructura del repositorio 6
7. Módulos del backend 6
7.1 Red (services/red) 6
7.2 Modelo exacto (services/exacto) 6
7.3 Algoritmo genético (services/genetico) 6
7.4 Sensibilidad y comparativo 7
8. API REST 7
9. Uso de la interfaz 8
10. Pruebas 9
11. Solución de problemas 9

1. Introducción

Este manual describe la arquitectura, la instalación, la configuración y el funcionamiento interno del sistema web de optimización logística desarrollado para EcoResiduos del Llano S.A.S. El sistema minimiza el costo semanal de transportar residuos agropecuarios desde 10 agroempresas hasta 4 plantas de transformación (Villavicencio, Restrepo, Granada y Puerto Gaitán).

Está dirigido a desarrolladores, evaluadores y personal técnico que necesite instalar, ejecutar, mantener o ampliar el sistema. Se asume conocimiento básico de Python, JavaScript, línea de comandos y Git.

2. Descripción general

El sistema resuelve el problema de transporte de la red logística con dos enfoques y permite compararlos:

- **Modelo exacto:** formulación del problema de transporte como programa lineal, resuelto con el método símplex (solver CBC mediante PuLP). A partir de la base óptima se obtienen los precios sombra con el método MODI.
- **Algoritmo genético (AG):** metaheurística que busca soluciones cercanas al óptimo con una población de soluciones que evoluciona mediante selección, cruce, mutación y reparación.

Además, ofrece parametrización de la red, análisis de sensibilidad, reporte comparativo, exportación a CSV e historial de escenarios y resultados en base de datos.

Tabla 1. Módulos del sistema

Módulo	Función
Parametrización	Agregar, editar y eliminar agroempresas y plantas; editar ofertas, capacidades, distancias y costos; validar datos y balancear la red.
Modelo exacto	Calcular el costo mínimo Z*, la asignación óptima, los precios sombra y verificar las restricciones.
Algoritmo genético	Ejecutar el AG con parámetros ajustables y mostrar el mejor costo, el gap y la curva de convergencia.
Sensibilidad	Calcular rangos de la base óptima y simular escenarios de combustible, oferta y cuellos de botella.
Comparativo	Comparar exacto y AG en costo, tiempo y gap, ante escenarios y al aumentar el tamaño de la red.
Historial	Guardar escenarios y resultados en PostgreSQL (opcional).

3. Arquitectura

El sistema sigue una arquitectura cliente-servidor de tres capas. El cliente es una aplicación web en React que se comunica por HTTP/JSON con una API REST en FastAPI. La API delega en servicios independientes por responsabilidad, que usan el solver CBC para el modelo exacto y leen o guardan los datos en archivos JSON o en PostgreSQL (Supabase).

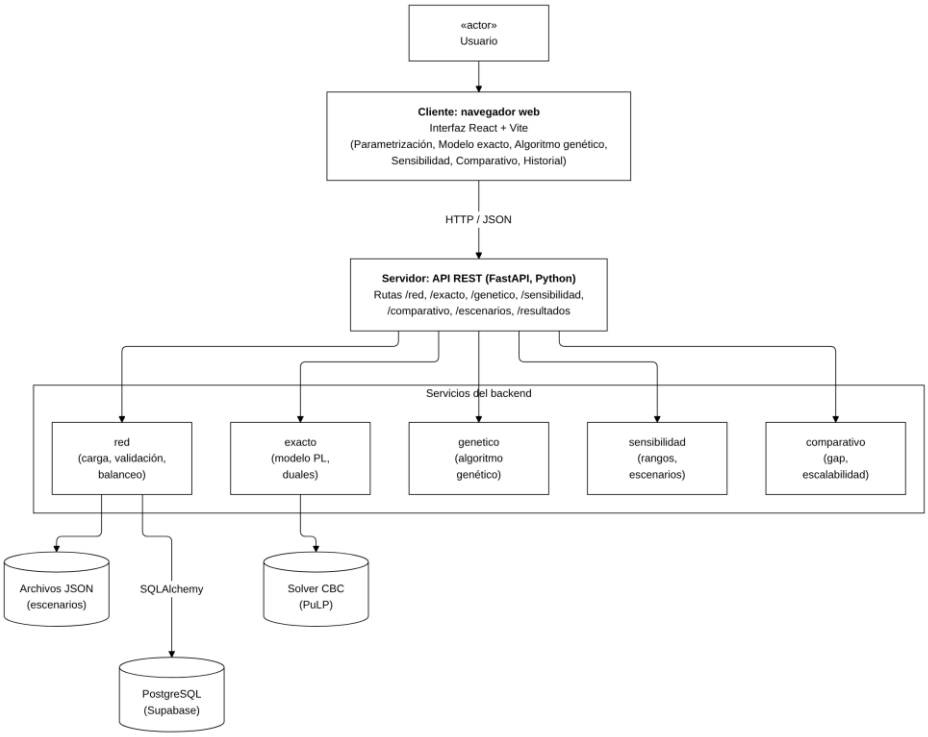


Figura 1. Arquitectura del sistema

Tabla 2. Tecnologías utilizadas

Capa	Tecnología	Uso
Interfaz	React 18.3, Vite 6.4, React Router 7	Vistas, formularios y navegación
Visualización	Chart.js 4.4, Leaflet 1.9	Gráficas, grafo y mapa de la red
API	Python 3.11+, FastAPI 0.115, Pydantic 2.9	Endpoints y validación de datos
Optimización	PuLP 2.9.0 con solver CBC	Modelo de programación lineal
Persistencia	SQLAlchemy 2.0, PostgreSQL (Supabase)	Escenarios e historial (opcional)
Pruebas	pytest 8.3, httpx	Pruebas automáticas del backend

4. Requisitos

- Python 3.11 o superior.
- Node.js 18 o superior y npm.
- Git, para clonar el repositorio.
- Navegador web moderno (Chrome, Edge o Firefox).
- Opcional: un proyecto en Supabase, para guardar el historial.

El sistema funciona en Windows, Linux y macOS.

5. Instalación y ejecución

5.1 Obtener el código

```
git clone https://github.com/LuciaVega-L/Optimizacion-Red-Logistica-Recepcion-y-Procesamiento-Residuos-Agropecuarios.git
```

5.2 Backend (API)

1. Entrar a la carpeta del backend y crear un entorno virtual.
2. Activar el entorno e instalar las dependencias.
3. Copiar el archivo de configuración de ejemplo.
4. Iniciar el servidor.

```
cd backend
python -m venv .venv
.venv\Scripts\activate          # Linux/Mac: source .venv/bin/activate
pip install -r requirements.txt
copy .env.example .env           # Linux/Mac: cp .env.example .env
uvicorn main:app --reload
```

La API queda disponible en `http://localhost:8000` y su documentación interactiva en `http://localhost:8000/docs`.

> **Nota:** use PuLP 2.9.0, como está fijado en requirements.txt. Con PuLP 3 o 4 el modelo exacto falla.

5.3 Frontend (interfaz)

```
cd frontend
npm install
npm run dev
```

La interfaz queda disponible en `http://localhost:5173`. El backend debe estar en ejecución.

5.4 Variables de entorno

Tabla 3. Variables de entorno (backend/.env y frontend/.env)

Variable	Valor por defecto	Descripción
FUENTE_DATOS	json	json: lee los escenarios de archivos. bd: usa PostgreSQL.
DATABASE_URL	—	URI de Supabase (Session pooler). Solo con FUENTE_DATOS=bd.
DATABASE_PASSWORD	—	Contraseña de la base de datos. Nunca se sube a GitHub.
CORS_ORIGENES	localhost:5173	Orígenes que pueden consumir la API.
CORS_LOCALHOST	true	Acepta localhost en cualquier puerto (desarrollo).
VITE_API_URL	http://localhost:8000	Dirección de la API (archivo frontend/.env).

5.5 Base de datos (opcional)

1. En Supabase, abrir SQL Editor y ejecutar `supabase/crear_base_datos_ecoresiduos.sql`.
2. En `backend/.env` poner FUENTE_DATOS=bd, DATABASE_URL y DATABASE_PASSWORD.

3. Cargar el caso base:

```
cd backend
python -m scripts.sembrar_bd
```

Sin base de datos el sistema funciona igual; solo no se guarda el historial.

6. Estructura del repositorio

Tabla 4. Estructura principal del repositorio

Carpeta	Contenido
backend/main.py	Punto de entrada de FastAPI (CORS y registro de rutas).
backend/app/api/routes/	Endpoints, uno por módulo.
backend/app/schemas/	Contratos de entrada y salida (Pydantic).
backend/app/services/	Lógica: red, exacto, genetico, sensibilidad, comparativo.
backend/app/db/	Conexión y modelos ORM (SQLAlchemy).
backend/data/escenarios/	dataConfig.json: caso base de la red.
backend/tests/	Pruebas automáticas con pytest.
frontend/src/pages/	Vistas: Inicio, Parametrización, Exacto, Genético, Sensibilidad, Comparativo.
frontend/src/components/	Matrices, grafo, mapa, gráficas e historial.
supabase/	Scripts SQL: tablas, vistas, funciones y seguridad.
docs/	Informe, diagramas, formulación y guía de la base de datos.

7. Módulos del backend

7.1 Red (services/red)

Carga el escenario, calcula el costo unitario de cada ruta, c_ij = c_u · d_ij + c_j^op (con c_u = 1.200 COP/km·t), valida los datos y detecta si la red está balanceada.

7.2 Modelo exacto (services/exacto)

Formula el problema de transporte como programa lineal. Si la oferta total no es igual a la capacidad total, agrega un nodo ficticio para balancear la red (en el caso base, la agroempresa E11 con 160 t/sem). El modelo se resuelve con el método símplex mediante el solver CBC. Con la solución, el módulo reconstruye la base óptima del problema de transporte y calcula los precios sombra u_i y v_j con el método MODI; con ellos verifica la optimalidad y obtiene los rangos de sensibilidad. Finalmente verifica las restricciones de oferta, capacidad y no negatividad.

7.3 Algoritmo genético (services/genetico)

Trabaja sobre la misma red balanceada que el modelo exacto. Cada individuo es una matriz de toneladas por ruta. La aptitud es f = 1 / (1 + Z), donde Z es el costo más una penalización cuadrática por exceder capacidades. Un operador de reparación mantiene todas las soluciones factibles.

Tabla 5. Parámetros del algoritmo genético

Parámetro	Valor por defecto	Ajustable desde la interfaz
Tamaño de población	100	Sí

Parámetro	Valor por defecto	Ajustable desde la interfaz
Número de generaciones	500	Sí
Tasa de cruce	0,8	Sí
Tasa de mutación	0,05	Sí
Selección	Torneo	Sí (torneo o ruleta)
Cruce	Un punto	Sí (uno o dos puntos)
Semilla	1	Sí (vacía = aleatoria)
Tamaño del torneo	3	No (constante TAM_TORNEO)
Número de élites	2	No (constante NUM_ELITES)

7.4 Sensibilidad y comparativo

El módulo de sensibilidad calcula los rangos de oferta, capacidad y costos que no alteran la base óptima, y simula escenarios porcentuales sin modificar la red original. El módulo comparativo calcula el gap = (Z_AG − Z_exacto) / Z_exacto × 100, los tiempos de ejecución y la escalabilidad en redes sintéticas.

8. API REST

Todas las rutas reciben y devuelven JSON. La lista completa, con ejemplos, está en http://localhost:8000/docs.

Tabla 6. Endpoints principales

Método	Ruta	Descripción
GET	`/red/escenarios`	Lista los escenarios disponibles.
POST	`/red/validar`	Errores de datos y estado del balance.
POST	`/exacto/resolver`	Z*, asignación, precios sombra y verificación.
POST	`/genetico/resolver`	Mejor costo, asignación, tiempo, factibilidad y convergencia.
POST	`/sensibilidad/rangos`	Rangos de costos, oferta y capacidad.
POST	`/sensibilidad/simular`	Simulación de escenarios (exacto y, opcional, AG).
POST	`/comparativo/reporte`	Exacto vs. AG: costo, tiempo y gap.
POST	`/comparativo/escalabilidad`	Comparación en redes de distintos tamaños.
GET/POST	`/escenarios`	Escenarios guardados (requiere base de datos).
GET/POST	`/resultados`	Historial de ejecuciones (requiere base de datos).

Los datos inválidos (valores negativos, campos vacíos, parámetros del AG fuera de rango o una selección distinta de torneo o ruleta) se rechazan con el código HTTP 422 y un mensaje que explica el error.

9. Uso de la interfaz

La barra lateral da acceso a cada módulo. El flujo recomendado es el siguiente:

1. **Parametrización:** revisar o editar la red y confirmar que no haya errores de datos.

2. **Modelo exacto:** presionar «Resolver» para obtener Z*, la asignación óptima y los flujos en la red (Figura 2).
3. **Algoritmo genético:** ajustar los parámetros y presionar «Ejecutar AG». Se muestran el mejor costo, el gap frente al exacto y la curva de convergencia (Figura 3).
4. **Sensibilidad:** simular los escenarios predefinidos; opcionalmente, resolverlos también con el AG.
5. **Comparativo:** revisar la comparación, medir la escalabilidad y exportar los resultados a CSV.

> **Nota:** para que el gap del AG aparezca, primero se debe resolver el modelo exacto en la misma sesión.

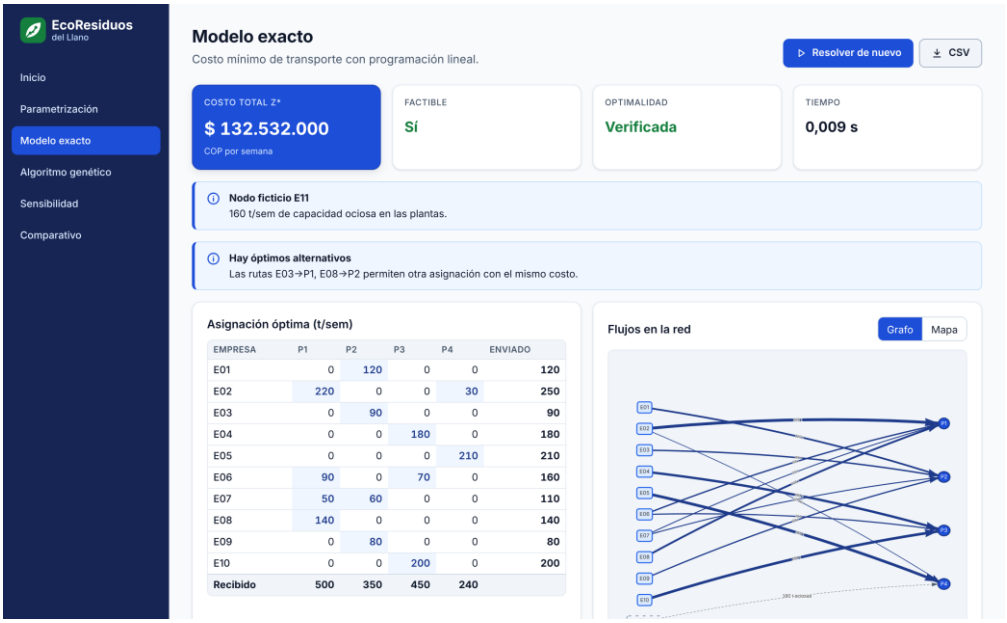


Figura 2. Vista del modelo exacto con el caso base

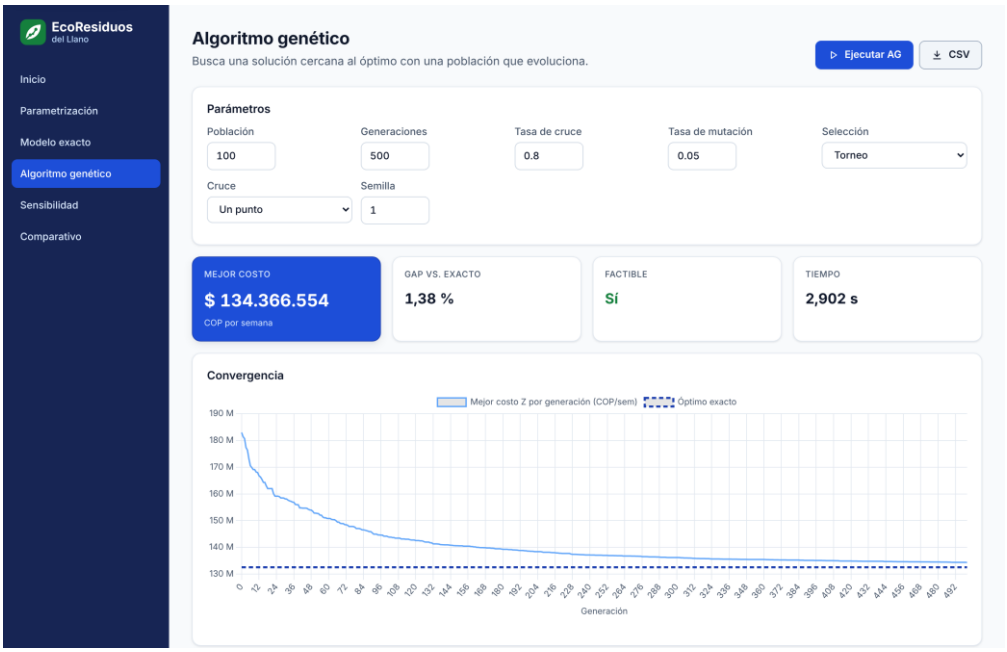


Figura 3. Vista del algoritmo genético con los parámetros por defecto

10. Pruebas

Las pruebas automáticas verifican el modelo exacto, el AG, la sensibilidad, el comparativo, la API y la base de datos. Se ejecutan desde la carpeta del backend:

```
cd backend
pytest
```

El resultado esperado es 63 pruebas aprobadas. Las 5 pruebas de base de datos se omiten si FUENTE_DATOS no es bd. La prueba del modelo exacto usa el caso base como referencia: Z* = 132.532.000 COP/semana y los precios sombra del informe.

11. Solución de problemas

Tabla 7. Problemas frecuentes

Problema	Solución
El modelo exacto falla al resolver	Verificar que esté instalada PuLP 2.9.0 (pip install -r requirements.txt).
La interfaz no carga datos (error de red o CORS)	Confirmar que el backend esté en ejecución y que VITE_API_URL apunte a él; revisar CORS_ORIGENES.
Error 503 al guardar en el historial	La base de datos no está activa: poner FUENTE_DATOS=bd y los datos de conexión en backend/.env.
Error 422 al resolver	Hay datos inválidos o parámetros fuera de rango; leer el mensaje que muestra la interfaz.
«Guardar en historial» deshabilitado	La red tiene cambios sin guardar; guardarla primero como escenario en Parametrización.
El puerto 8000 o 5173 está ocupado	Cerrar el proceso que lo usa o iniciar con otro puerto (uvicorn --port 8001).